

实验报告

1. 实验概述 (Introduction)

1.1 实验背景

在此前的实验中，我们已经实现了物理内存管理、虚拟内存管理、内核线程、用户进程、CPU调度以及进程间的同步互斥机制。然而，目前的操作系统还缺乏数据持久化的能力。一旦系统断电或重启，内存中的所有数据都会丢失。为了使操作系统具备实用性，必须引入**文件系统 (File System)**。

文件系统不仅负责将数据持久化存储在磁盘等介质上，还通过**虚拟文件系统 (VFS)** 向上层用户程序提供统一的访问接口（如 `open`, `read`, `write`, `close`），屏蔽了底层具体文件系统（如 SFS, FAT, EXT4）和硬件设备的差异。

1.2 实验目的

本实验的主要目的是在 uCore 操作系统中实现一个简单的文件系统（SFS），并将其与 VFS 结合，最终使操作系统能够从磁盘加载并执行应用程序。具体目标包括：

- 理解 VFS/SFS 架构**：掌握 uCore 中虚拟文件系统和简单文件系统的设计与实现。
- 实现文件 I/O**：编写底层的文件读写函数，处理磁盘块的索引与缓冲。
- 实现进程加载**：修改进程创建逻辑，使其能够解析 ELF 格式的磁盘文件并加载到内存执行。
- 扩展机制设计**：设计 UNIX 风格的管道 (Pipe) 和链接 (Hard/Soft Link) 机制。

2. 实验原理

2.1 虚拟文件系统 (VFS)

uCore 采用了 VFS 架构，核心数据结构包括：

- inode (索引节点)**：文件系统中的基本对象，代表一个文件或目录。包含文件类型、大小、权限以及指向具体文件系统数据的指针。
- file (打开文件结构)**：代表进程打开的一个文件实例，包含当前的读写偏移量、访问模式等。
- superblock (超级块)**：描述整个文件系统的元数据。
- dentry (目录项)**：用于路径解析，建立文件名到 inode 的映射（uCore 中简化处理，直接集成在 inode 查找中）。

2.2 Simple File System (SFS)

SFS 是 uCore 自定义的一个简单文件系统，其磁盘布局如下：

- Super Block**：位于第 0 块，存储文件系统总块数、未使用块数等信息。
- Root Dir**：根目录的 inode 节点。
- FreeMap**：位图，用于记录磁盘块的分配状态。
- Data Blocks**：实际存储 inode 和文件数据的区域。

SFS 使用索引结构来管理大文件：

- Direct Blocks**：直接指向数据块 (0-11)。
- Indirect Blocks**：一级间接索引 (12)。
- Double Indirect Blocks**：虽然定义支持，但在基础实验中通常只用到一级索引。

3. 练习1：完成读文件操作的实现

3.1 任务分析

本练习要求在 `kern/fs/sfs/sfs_inode.c` 中实现 `sfs_io_nolock` 函数。该函数是 SFS 读写操作的核心，负责将文件逻辑偏移量 (offset) 映射到物理磁盘块，并通过缓冲区进行数据传输。

难点：

1. **块对齐处理**：读写操作可能不是从块的起始位置开始，也不是在块的结束位置结束。需要处理“首部非对齐”、“中间完整块”、“尾部非对齐”三种情况。
2. **索引查找**：必须使用 `sfs_bmap_load_nolock` 函数将文件的逻辑块号 (index) 转换为磁盘上的物理块号 (ino)。

3.2 代码实现与解释

函数原型：

```
static int
sfs_io_nolock(struct sfs_fs *sfs, struct sfs_inode *sin, void *buf, off_t
offset, size_t *alen, bool write)
```

实现逻辑详解：

1. **初始化与边界检查**：
首先计算结束位置 `endpos`。对于读操作，不能超过文件大小；对于写操作，可能需要扩展文件。
2. **计算起始块信息**：
 - `blkoff = offset % SFS_BLKSIZE`：计算在第一个块内的偏移。
 - `nblks`：计算涉及的总块数。
3. **主循环 (Buffer Loop)**：
只要还有数据需要传输 (`len > 0`)，就进入循环：
 - **步骤A：获取物理块号**
调用 `sfs_bmap_load_nolock(sfs, sin, blkno, &ino)`。
如果 `ino == 0`：
 - 如果是写操作：分配新块 (`sfs_block_alloc`)。
 - 如果是读操作：读取空洞（即文件该部分未分配物理块），通常填零。
 - **步骤B：确定当前块的传输长度**
`size = (nblks == 0) ? (endpos % SFS_BLKSIZE) : SFS_BLKSIZE`。
如果是第一块且不对齐，`size -= blkoff`。
 - **步骤C：执行 I/O**
利用 `sfs_buf_op` 函数，该函数封装了磁盘缓存的操作。
 - 如果 `blkoff == 0` 且 `size == SFS_BLKSIZE`（整块读写）：直接对该块进行覆盖写或读。
 - 如果是部分读写：
 - 写操作：先将磁盘块读入 buffer，修改部分内容，再标记为脏。
 - 读操作：将磁盘块读入 buffer，拷贝所需部分到用户 buf。
 - **步骤D：更新指针**
`len -= size, buf += size, offset += size, blkno++`，`blkoff = 0`（后续块都是从0开始）。

核心代码片段（读部分）：

```

if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
    goto out;
}
/* 处理逻辑块号 blkno 到物理块号 ino 的映射 */
/* 计算本轮需要操作的数据量 size */

if (ino == 0) {
    // 读空洞，填0
    memset(buf, 0, size);
} else {
    // 通过 buffer cache 读数据
    ret = sfs_buf_op(sfs, buf, size, ino, blkoff);
}

```

3.3 实验分析

为什么需要 `sfs_io_nolock` 而不是直接读盘？

因为磁盘读写的基本单位是扇区（或块，通常4KB）。如果用户只想读 10 个字节，操作系统不能直接指令磁盘“只读10字节”。必须将包含这10字节的整个 4KB 块读入内核内存（Buffer Cache），然后将这10字节 `memcpy` 给用户。`sfs_io_nolock` 正是处理这种“逻辑流”到“物理块”转换的关键。

4. 练习2：完成基于文件系统的执行程序机制

4.1 任务分析

在 Lab 5 中，用户程序是直接链接在内核镜像中的。在 Lab 8，我们需要实现从文件系统中读取 ELF 格式的可执行文件，并将其加载到内存中执行。这主要涉及修改 `kern/process/proc.c` 中的 `load_icode` 函数。

改写目标：

原 `load_icode` 接收的是内存中的二进制指针；新 `load_icode` 接收的是文件句柄（fd）。

4.2 实现步骤与代码逻辑

1. 建立内存空间

调用 `mm_create` 创建新的内存管理结构 `mm`，并调用 `setup_pgdir` 分配项目录表。

2. 解析 ELF Header

由于文件在磁盘上，不能直接指针访问。

- 调用 `load_icode_read`（封装 `sys_read` 或底层 `VOP_READ`），读取文件的前 `sizeof(struct elfhdr)` 字节到栈上的 `struct elfhdr __elf` 变量中。
- 检查 Magic Number 确认是合法的 ELF 文件。

3. 遍历 Program Headers

根据 `elf.e_phoff` 找到程序头表的位置。循环读取每一个 `struct proghdr ph`。

4. 加载 Segment

对于类型为 `ELF_PT_LOAD` 的段：

- **内存分配：**调用 `mm_map`，根据 `ph.p_memsz` 和 `ph.p_va` 建立虚拟地址空间的映射（设置 VMA）。
- **数据读取：**
 - 计算段在文件中的偏移 `ph.p_offset`。

- 调用 `load_icode_read` 将文件内容读取到对应的虚拟地址 `ph.p_va` 中。
- **注意**：这里需要处理 Page Fault。由于我们已经 `mm_map` 了，但没有实际分配物理页，直接读入可能会触发缺页中断。在 uCore 的实现中，通常会手动分配页或者利用现有的缺页处理机制。为了简化，可以直接对这些地址进行 `memcpy`（如果处于内核态可以直接访问用户空间映射）或者使用特定的 API 将文件内容复制到分配好的物理页中。
- **BSS 处理**：如果 `p_memsz > p_filesz`，说明存在 BSS 段，需要将多出的部分清零。

5. 设置用户栈

分配用户栈空间（通常是 `USTACKTOP` 之下），并映射物理页。

6. 处理参数 (argc, argv)

这是 Lab 8 的一个重要新增点。用户程序（如 `sh`）需要接收参数。

- 计算参数的总长度，确保栈空间足够。
- 将参数字符串从内核空间拷贝到用户栈顶。
- 在栈上构建 `argv` 指针数组，指向这些字符串在用户栈中的地址。
- 设置 `tf->tf_esp` 为调整后的栈顶。

7. 上下文切换

设置 `tf->tf_eip` 为 ELF 的入口地址 `elf.e_entry`。设置 `ret = 0` 表示成功。

4.3 结果验证

执行 `make qemu`。如果系统启动后进入 shell 界面，且能够执行 `ls`, `hello` 等命令，说明：

1. SFS 挂载成功。
2. `fork` 机制正常。
3. `exec` (即 `load_icode`) 成功从磁盘读取了 `sh` 并执行。

5. 扩展练习 Challenge 1: UNIX PIPE 机制设计方案

5.1 设计概述

管道 (Pipe) 是 UNIX 系统中最古老且最重要的 IPC (进程间通信) 机制之一。它允许一个进程的输出直接作为另一个进程的输入。

在 uCore 中，管道可以被设计为一种特殊的“内存文件”。它没有对应的磁盘 inode，而是存在于内存缓冲区中。

5.2 数据结构设计

我们需要定义一个核心结构体来管理管道的状态，包括缓冲区、读写指针和同步原语。

```
#define PIPE_SIZE 4096

struct pipe_inode_info {
    char *base;           // 指向内核分配的环形缓冲区
    unsigned int head;     // 写入位置 (producer index)
    unsigned int tail;     // 读取位置 (consumer index)
    unsigned int nreaders; // 读者计数
    unsigned int nwriters; // 写者计数

    semaphore_t mutex;     // 互斥锁，保护 head/tail 操作
    semaphore_t wait_data; // 读进程等待数据的信号量 (空时阻塞)
    semaphore_t wait_space; // 写进程等待空间的信号量 (满时阻塞)
```

```
struct proc_struct *reader_proc; // 可选：用于唤醒特定进程
struct proc_struct *writer_proc;

};
```

5.3 接口设计 (语义)

1. pipe(int fd[2])

- **语义**：创建一个管道，分配两个文件描述符。fd[0] 用于读，fd[1] 用于写。
- **实现**：
 1. 分配一个新的 inode，标记类型为 SFS_TYPE_PIPE（或者专门的 PIPE FS）。
 2. 分配 pipe_inode_info 并关联到 inode。
 3. 在当前进程的 files_struct 中分配两个 file 对象，分别指向这个 inode，但打开模式分别为 O_RDONLY 和 O_WRONLY。

2. read(pipe_fd, buf, size)

- **语义**：从管道读取数据。如果缓冲区空，则阻塞。
- **同步逻辑**：
 1. 获取 mutex。
 2. while (is_empty):
 - 如果 nwriters == 0 (写端全部关闭)，返回 EOF (0)。
 - 释放 mutex，P操作 wait_data (睡眠)，重新获取 mutex。
 3. 读取数据，更新 tail。
 4. V操作 wait_space (唤醒写者)。
 5. 释放 mutex。

3. write(pipe_fd, buf, size)

- **语义**：向管道写入数据。如果缓冲区满，则阻塞。
- **同步逻辑**：
 1. 获取 mutex。
 2. while (is_full):
 - 如果 nreaders == 0 (读端关闭)，发送 SIGPIPE 信号或返回错误。
 - 释放 mutex，P操作 wait_space，重新获取 mutex。
 3. 写入数据，更新 head。
 4. V操作 wait_data (唤醒读者)。
 5. 释放 mutex。

4. close(pipe_fd)

- **语义**：关闭管道一端。
- **逻辑**：
 - 减少 nreaders 或 nwriters。
 - 如果计数变为 0，需释放所有等待在另一端的进程（V操作所有信号量），让它们醒来处理 EOF 或 Broken Pipe。
 - 当 nreaders == 0 && nwriters == 0 时，释放缓冲区和结构体内存。

5.4 互斥与死锁预防

- 使用 `mutex` 保证对 `head` 和 `tail` 指针修改的原子性。
- 使用条件变量（在 uCore 中可用信号量模拟）来实现“空等待”和“满等待”。
- **死锁预防**：必须注意 `lock` 的粒度。在 `copy_to_user` 或 `copy_from_user` 期间（可能会发生缺页中断），最好不要持有自旋锁；但如果持有的是互斥锁（Semaphore），则允许睡眠。

6. 扩展练习 Challenge 2: UNIX 软/硬链接机制设计方案

6.1 设计概述

- **硬链接 (Hard Link)**：不同的文件名指向同一个 inode。只有当引用计数降为 0 时，实际数据才被删除。
- **软链接 (Soft Link / Symlink)**：一个特殊的文件，其内容是指向另一个文件的路径字符串。

6.2 硬链接设计

数据结构修改：

在 `struct sfs_disk_inode` (磁盘上的 inode 结构) 中，必须确保有一个字段 `uint16_t nlinks`。uCore 现有的 SFS 定义中可能已有保留字段或复用字段，需明确定义为链接计数。

接口设计：

1. `link(old_path, new_path)`

- **语义**：为 `old_path` 创建一个新的硬链接 `new_path`。
- **实现步骤**：
 1. 解析 `old_path` 获得 `old_inode`。
 2. 检查 `old_inode` 是否为目录（通常不允许对目录建立硬链接以防环路，除非是超级用户）。
 3. 解析 `new_path` 的父目录 `new_parent_inode`。
 4. 在 `new_parent_inode` 中创建一个新的目录项 (entry)，名称为 `new_path` 的文件名，但对应的 inode 编号设为 `old_inode->ino`。
 5. **关键同步**：加锁 `old_inode`，`old_inode->nlinks++`，标记 inode 为 dirty 并写回磁盘。
 6. 解锁。

2. `unlink(path)`

- **语义**：删除文件名。
- **实现步骤**：
 1. 查找 `path` 对应的目录项和 `target_inode`。
 2. 从父目录中删除该目录项。
 3. **关键同步**：加锁 `target_inode`，`target_inode->nlinks--`。
 4. 如果 `nlinks == 0` 且当前没有进程打开该文件（需检查内存中的 open count），则释放该 inode 占用的所有数据块 (`sfs_truncate`) 并释放 inode 本身。
 5. 写回磁盘。

6.3 软链接设计

数据结构修改：

需要定义一种新的文件类型：`SFS_TYPE_LINK`。

接口设计：

1. `symlink(target_path, link_path)`

- **语义：**创建一个路径为 `link_path` 的软链接，指向 `target_path`。
- **实现步骤：**
 1. 在 `link_path` 处创建一个新文件。
 2. 将该文件的 inode 类型设为 `SFS_TYPE_LINK`。
 3. 将 `target_path` 字符串作为文件内容写入该文件的数据块中。

2. `readlink(path, buf, size)`

- **语义：**读取软链接本身的内容（即目标路径）。
- **实现：**打开文件，如果是 `SFS_TYPE_LINK`，读取数据块内容返回。

3. 路径解析的修改 (`vfs_lookup`)

- 这是软链接最复杂的地方。
- 在解析路径时，如果遇到中间某个节点是 `SFS_TYPE_LINK`：
 1. 读取其内容（目标路径）。
 2. 如果目标路径是绝对路径（以 `/` 开头），则从根目录重新开始解析。
 3. 如果是相对路径，则相对于当前目录解析。
 4. **死循环保护：**必须维护一个计数器（如 `MAX_SYMLINKS = 8`）。如果递归解析软链接次数超过限制，返回 `ELOOP` 错误。

6.4 同步互斥分析

- **Race Condition：**两个进程同时对同一个文件执行 `link` 或 `unlink`。
- **解决方案：**
 - inode 级别的锁 (`sin->sem`) 必须在修改 `nlinks` 期间持有。
 - 目录级别的锁：在目录中添加或删除 entry 时，必须锁住父目录的 inode。
 - **死锁风险：**例如 `rename` 操作可能涉及两个目录，需要按照 inode 编号顺序加锁以避免死锁。

7. 实验总结 (Conclusion)

7.1 实验成果

通过本次实验，我们成功地将 uCore 从一个仅能在内存中运行简单逻辑的内核，升级为一个具备持久化存储能力、支持文件操作、能够从磁盘加载标准 ELF 程序运行的完整操作系统。

1. **SFS 的实现**使我们深入理解了 inode、bitmap 和数据块在磁盘上的物理组织形式。
2. **VFS 的抽象**展示了操作系统如何通过统一接口解耦文件系统实现与用户调用。
3. **exec/load_icode 的重构**打通了文件系统与进程管理之间的桥梁。

7.2 遇到的问题与解决

- **缓冲区管理**: 在实现 `sfs_io_nolock` 时, 对于边界对齐的计算非常容易出错。通过画图分析首尾块的偏移量, 并仔细调试边界条件 (如 `offset=4095, len=2`) , 解决了数据错位问题。
- **参数传递**: 在 `load_icode` 中构建用户栈参数时, 字符串拷贝的指针运算较为复杂, 容易导致 Page Fault。通过仔细核对栈顶指针 `esp` 的移动逻辑解决。